

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

D. Sravanthi (VTU26109)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Job Portal Management System" by D. Sravanthi (VTU26109) has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr. S. Hemamalini

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of the Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(D.Sravanthi)

Date:06/04/2026

ABSTRACT

The Job Portal Management System: Enterprise Recruitment & Placement Platform is a modern full stack web application developed to simplify and enhance the campus recruitment and placement process through a centralized digital system. The platform provides secure authentication for students and administrators, job application management, recruitment analytics, company management, notifications, login tracking, and dashboard systems. Students can explore job opportunities, apply for jobs, track applications, and manage their profiles, while administrators can monitor recruitment activities, manage companies and job postings, and analyze placement statistics through interactive dashboards. The project is developed using React.js, Tailwind CSS, Node.js, Express.js, and Firebase with JWT authentication and bcrypt password hashing to ensure security and scalability. The platform demonstrates modern web development, responsive UI design, secure database integration, and enterprise-level recruitment management suitable for real-world placement systems.

Keywords: Full Stack Web Development, Campus Recruitment System, React.js, Node.js, Firebase, JWT Authentication, Recruitment Analytics

LIST OF FIGURES

1.1	System Framework Architecture	8
3.1	Main Page	13
4.1	Project Architecture of Job Portal Management System	15
4.2	Data Flow Diagram of Job Portal Management System	16
4.3	Registration Page	17
4.4	Details for Job Application	18
4.5	Login details	19
4.6	Students Applied	20
5.1	Students Shortlisted	22

LIST OF TABLES

5.1	Comparison of Recruitment Platform Features	22
7.1	Mapping of Project to Complex Engineering Problem (CEP)	25
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	26
7.3	SDG Mapping for the Project	26

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
JSON	JavaScript Object Notation
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	1
LIST OF TABLES	2
LIST OF ACRONYMS AND ABBREVIATIONS	3
1 INTRODUCTION	6
1.1 Introduction	6
1.2 Aim of the Project	6
1.3 Scope of the Project	7
1.4 Framework	7
2 SURVEY OF SIMILAR APPLICATION IN MARKET	9
3 FRAMEWORK DESCRIPTION	11
3.1 Frontend Implementation	11
3.2 Backend Implementation	12
3.3 Database Implementation	13
4 METHODOLOGIES	15

4.1	Project Architecture	15
4.2	DataFlow Diagram	16
4.3	Module 1: Authentication Module	17
4.4	Module 2: Recruitment Management Module	18
4.5	Module 3: Analytics and Monitoring Module	18
5	RESULTS AND DISCUSSIONS	21
6	CONCLUSION AND FUTURE ENHANCEMENTS	23
6.1	Conclusion	23
6.2	Future Enhancements	24
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	25
7.1	Complex Engineering Problem (CEP)	25
7.2	Complex Engineering Activities (CEA)	26
7.3	SDG Mapping	26
	References	27
8	SOURCE CODE	27
8.1	Source Code	27
	References	31

Chapter 1

INTRODUCTION

1.1 Introduction

The Job Portal Management System: Enterprise Recruitment & Placement Platform is a modern full stack web application developed to improve and digitalize the campus recruitment process. Traditional placement systems often depend on manual operations and disconnected communication methods, making recruitment management difficult and inefficient. The proposed platform provides a centralized system where students can explore job opportunities, apply for jobs, track applications, and manage their profiles through an interactive dashboard. Administrators can manage companies, job postings, student approvals, analytics, reports, and recruitment activities efficiently. The platform is developed using React.js, Node.js, Express.js, and Firebase with secure JWT authentication and responsive dashboard systems to provide a professional and scalable recruitment environment.

1.2 Aim of the Project

The main aim of the project is to develop a secure and modern recruitment management platform that simplifies campus placement activities for both students and administrators. The system aims to provide centralized job management, secure au-

thentication, analytics dashboards, company management, application tracking, and responsive user interfaces to improve the efficiency of recruitment and placement processes.

1.3 Scope of the Project

The scope of the project includes student registration, secure login systems, company management, job posting management, application tracking, recruitment analytics, notifications, login monitoring, and dashboard systems. The platform supports both student and admin modules and can further be enhanced with mobile applications, cloud deployment, AI-based recommendations, and advanced analytics systems in the future.

1.4 Framework

The Job Portal Management System platform follows a full stack web application architecture consisting of frontend, backend, and database layers. The frontend is developed using React.js and Tailwind CSS to provide responsive and interactive user interfaces. The backend is developed using Node.js and Express.js to manage APIs, authentication, and business logic. Firebase is used as the relational database for storing students, companies, jobs, applications, and analytics data. JWT authentication and bcrypt password hashing are implemented to ensure secure access and data protection within the platform.

Smart CareerHub

Enterprise Recruitment & Placement Platform

System Architecture

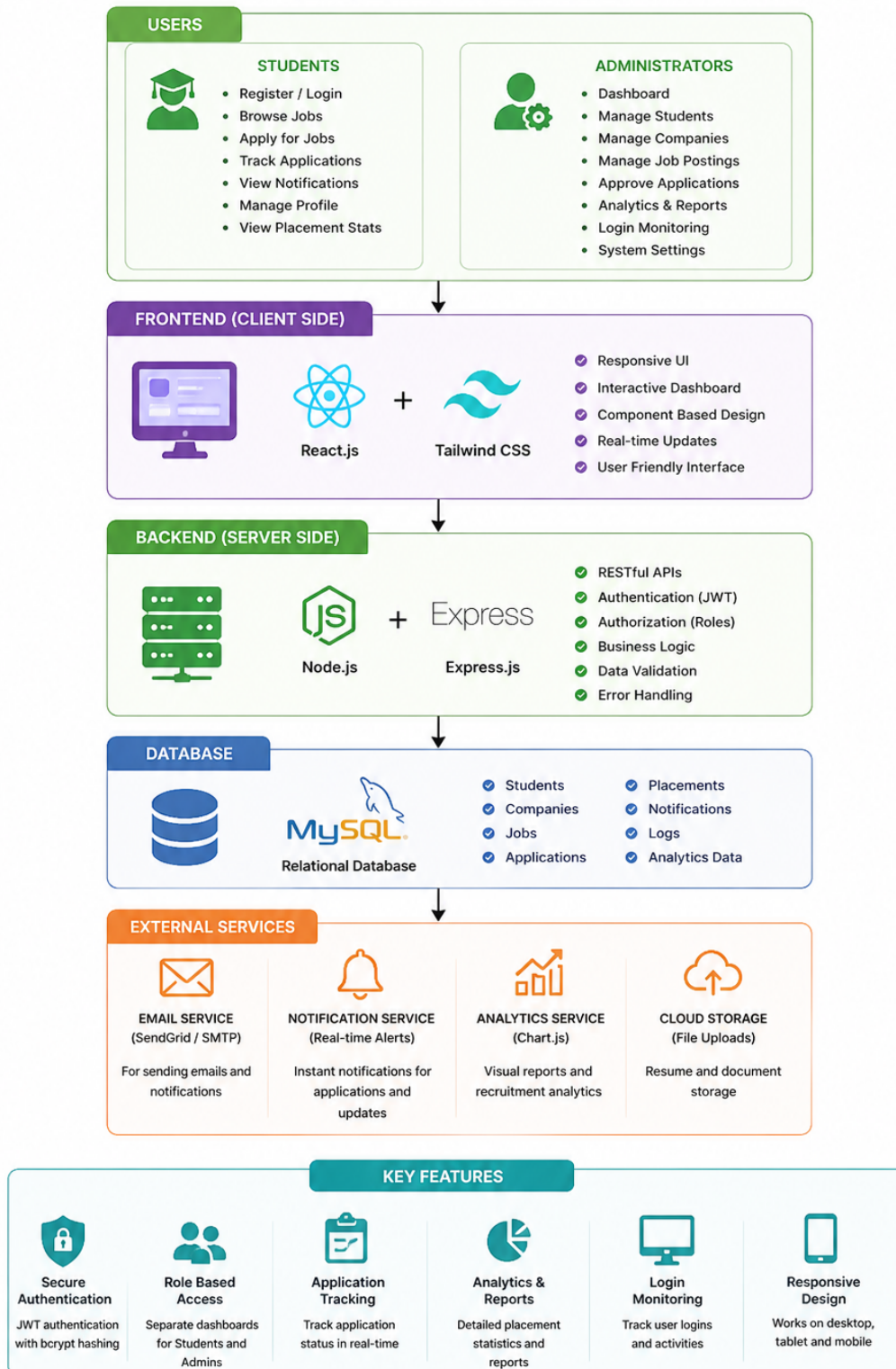


Figure 1.1: System Framework Architecture

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The rapid growth of digital technologies has led to the development of several online recruitment and placement platforms that simplify hiring and placement activities. Many organizations and educational institutions use web-based recruitment systems to manage student placements, company hiring processes, and job applications efficiently. Existing platforms such as LinkedIn, Internshala, Naukri, Indeed, and campus recruitment portals provide job listings, profile management, and recruitment services for students and professionals.

LinkedIn is one of the most popular professional networking and recruitment platforms that allows users to create professional profiles, connect with recruiters, and apply for jobs. However, it mainly focuses on professional networking and does not provide institution-specific placement management features.

Internshala is widely used for internships and student job opportunities. It provides internship listings, application systems, and skill development programs. However, it lacks advanced campus recruitment analytics and institution-based student approval workflows.

Naukri and Indeed are large-scale job portals that provide recruitment and hiring

services for companies and job seekers. These platforms support job search, resume uploads, and recruitment management but are not specifically designed for educational institution placement systems.

Many college placement systems currently used in institutions are basic management portals with limited dashboard analytics, weak authentication systems, poor UI design, and minimal security monitoring features. Most traditional systems also lack centralized analytics dashboards, login tracking, responsive interfaces, and enterprise-level placement management capabilities.

The proposed Job Portal Management System : Enterprise Recruitment & Placement Platform overcomes these limitations by providing a centralized and modern recruitment management system with secure authentication, responsive dashboards, analytics visualization, company management, application tracking, notifications, login monitoring, and role-based access control. The platform combines the features of modern recruitment systems with institution-specific placement management capabilities to provide an efficient and scalable solution.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Job Portal Management System platform was developed using React.js and Tailwind CSS to provide a responsive and modern user interface. The development environment was configured using Node.js, npm, and Vite. Required frontend dependencies such as React Router DOM, Axios, Framer Motion, and Chart.js were installed to support routing, API integration, animations, and analytics visualization. Visual Studio Code was used as the primary development environment by Dinesh for implementing and managing the frontend architecture.

3.1.2 Structure of the Project

The frontend project follows a component-based architecture where the application is divided into reusable components, pages, layouts, and utility modules. The project structure contains separate folders for authentication pages, dashboards, job management pages, analytics pages, reusable UI components, API services, and routing configuration. This modular structure improves maintainability, scalability, and code reusability within the platform.

3.1.3 Execution Procedure

The frontend application is executed using the Vite development server. After installing the required dependencies using `npm install`, the application is started using the `npm run dev` command. The frontend communicates with backend APIs through Axios requests and renders dynamic content such as dashboards, analytics, job listings, notifications, and application tracking systems.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the Smart CareerHub platform was developed using Node.js and Express.js. The backend environment was configured using npm packages including Express, Firebase, JWT, bcrypt, dotenv, cors, and nodemon. The backend APIs were designed to handle authentication, company management, job management, student approvals, notifications, analytics, and database operations securely and efficiently.

3.2.2 Structure of the Project

The backend project follows a modular architecture consisting of routes, controllers, middleware, configuration files, and database utilities. Separate API modules are implemented for authentication, student operations, admin operations, job management, analytics, and notifications. Middleware functions are used for JWT authentication, role verification, validation, and error handling to ensure secure and scalable backend operations.

3.2.3 Execution Procedure

The backend server is executed using Node.js and Express.js. After installing dependencies using `npm install`, the backend server is started using `npm run dev` or `nodemon server.js`. The backend establishes a secure connection with the Fire-

base database and exposes RESTful APIs for frontend communication. Authentication, CRUD operations, analytics queries, and application management are processed through the backend server.

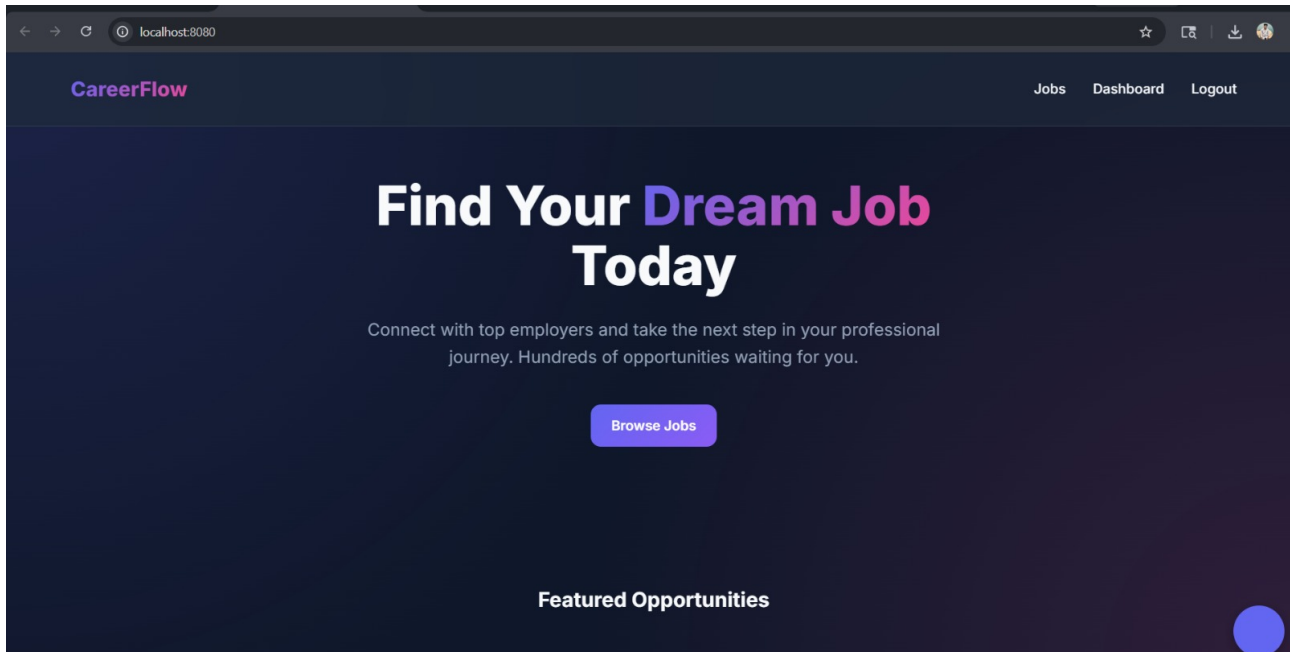


Figure 3.1: Main Page

3.3 Database Implementation

3.3.1 Database Configuration

The Job Portal Management System platform uses Firebase as the relational database management system for storing application data securely and efficiently. The database was configured locally using Firebase. Environment variables were used to establish secure database connections between the backend server and the Firebase database.

3.3.2 Schema Structure

The database schema was designed to support authentication, recruitment management, analytics, notifications, and application tracking. Relationships were established using primary keys and foreign keys to maintain data integrity and relational

consistency across multiple modules within the platform.

3.3.3 Tables Created

The following tables were created in the Firebase database for the Job Portal Management System platform:

- admins
- students
- companies
- jobs
- applications
- notifications
- login_logs
- security_logs
- reports

These tables store user information, authentication details, company records, job postings, analytics data, login activities, notifications, and application tracking information required for the recruitment platform.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Job Portal Management System platform follows a three-tier architecture using React.js, Node.js, Express.js, and Firebase to provide secure, scalable, and efficient recruitment management services.

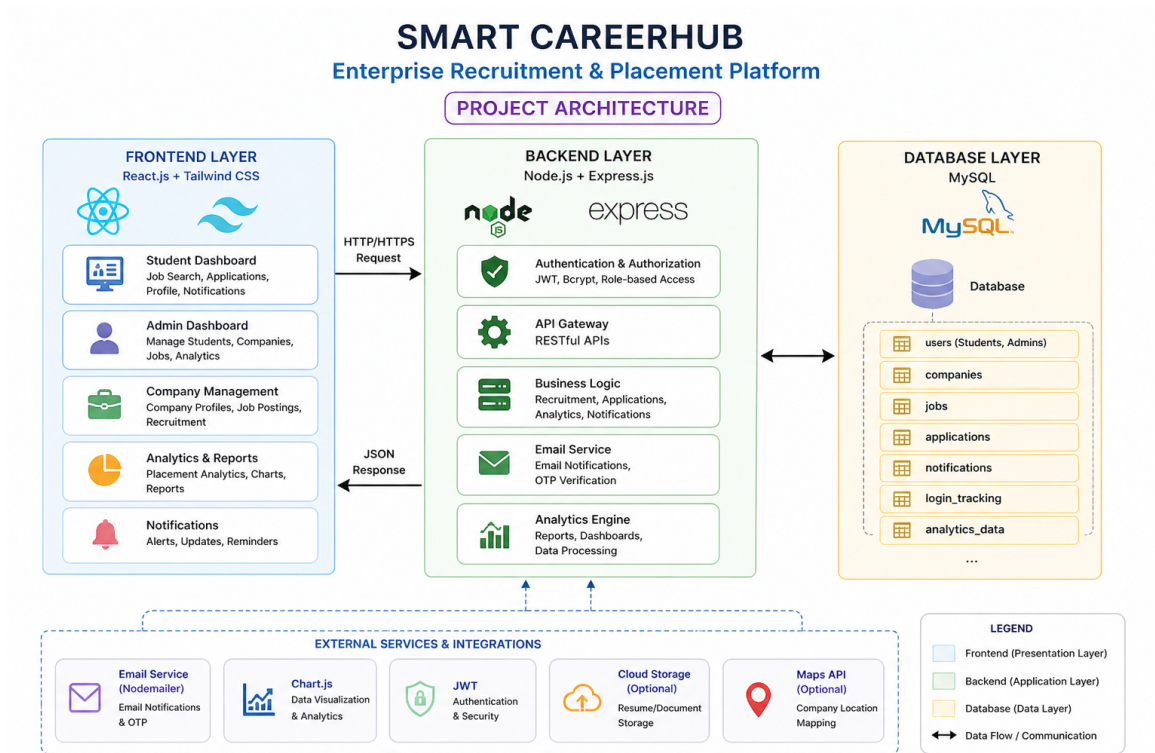


Figure 4.1: Project Architecture of Job Portal Management System

4.2 DataFlow Diagram

The data flow within the Job Portal Management System platform begins with students and administrators interacting with the frontend interface. User requests are sent to the backend server through RESTful APIs. The backend processes authentication, validates requests, manages business logic, and communicates with the Firebase database to store or retrieve information. The processed data is then returned to the frontend and displayed dynamically through dashboard systems, analytics pages, job management modules, and notification systems. This structured data flow ensures smooth communication between all components of the platform.

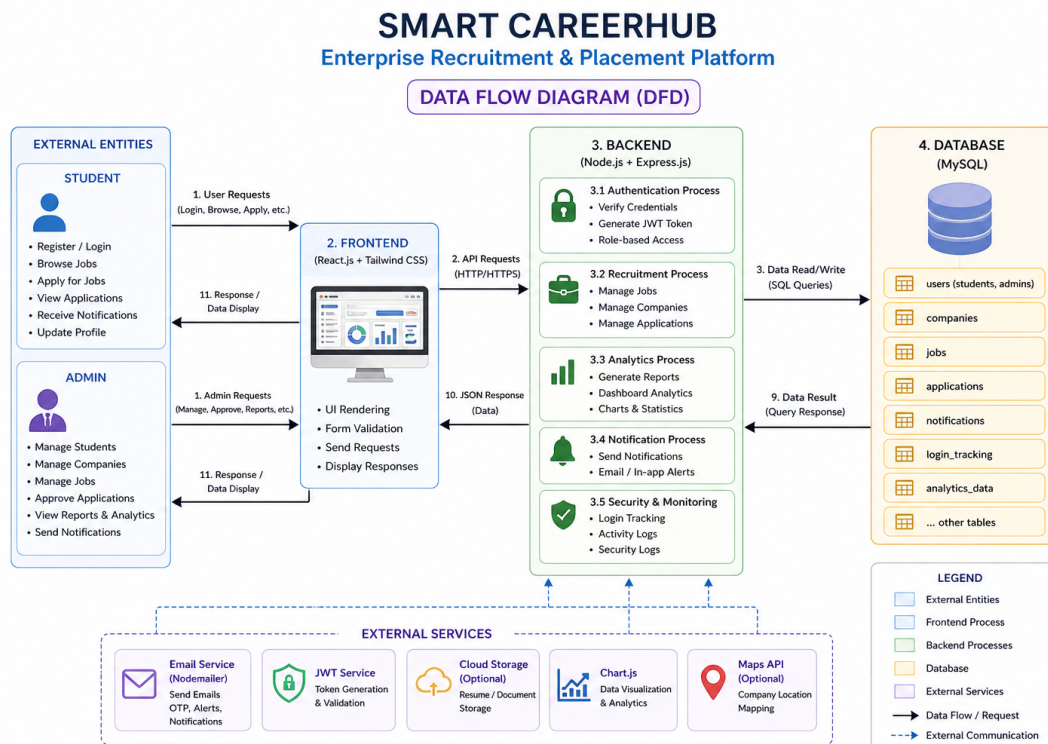
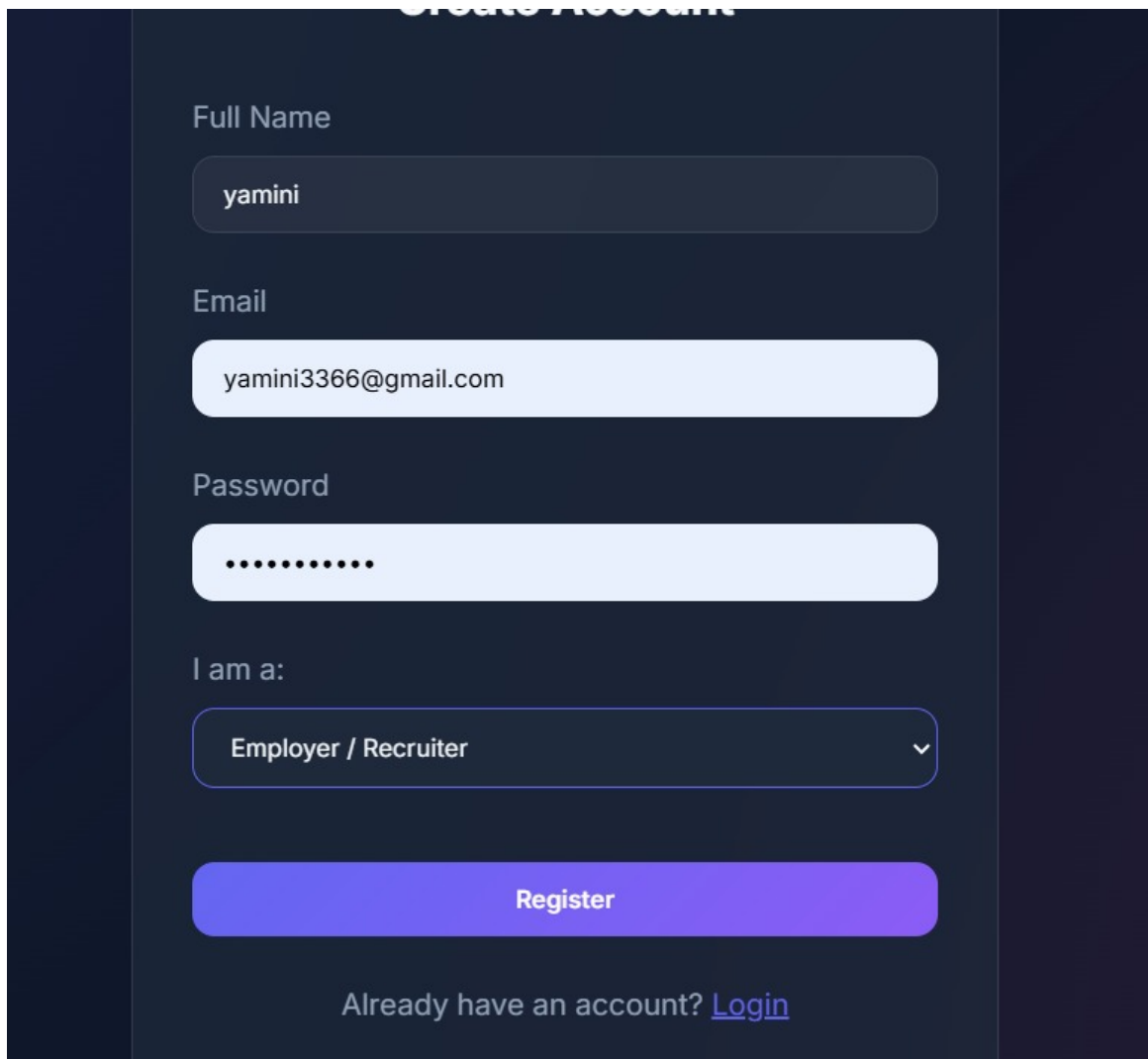


Figure 4.2: Data Flow Diagram of Job Portal Management System

4.3 Module 1: Authentication Module

The authentication module manages secure login and signup functionality for both administrators and students. JWT authentication and bcrypt password hashing are implemented to ensure secure access management. Students are required to register and wait for admin approval before accessing the platform. Role-based authentication is used to protect dashboards and APIs from unauthorized access.



The image shows a registration form titled "Create Account" on a dark blue background. The form contains the following fields and elements:

- Full Name:** A text input field with the value "yamini".
- Email:** A text input field with the value "yamini3366@gmail.com".
- Password:** A password input field with masked characters ".....".
- I am a:** A dropdown menu with the selected option "Employer / Recruiter" and a downward arrow icon.
- Register:** A large, rounded, light blue button.
- Footer:** The text "Already have an account? [Login](#)" in a light blue font.

Figure 4.3: Registration Page

4.4 Module 2: Recruitment Management Module

The recruitment management module handles company management, job postings, application tracking, and student recruitment activities. Administrators can add companies, manage job openings, approve students, and monitor recruitment statistics. Students can browse jobs, apply for opportunities, save jobs, and track their application status through the dashboard.

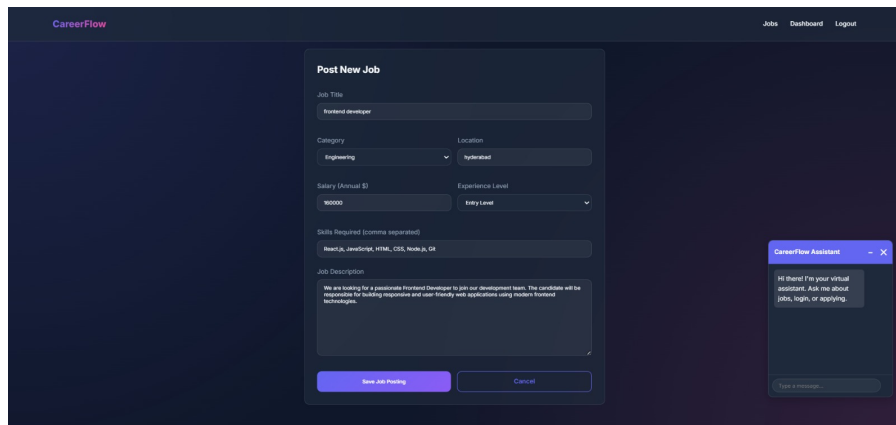
The screenshot shows a web application interface for 'CareerFlow'. At the top, there's a navigation bar with 'Jobs', 'Dashboard', and 'Logout' links. The main content area is titled 'Post New Job'. It contains several input fields: 'Job Title' (with 'Backend Developer' entered), 'Category' (a dropdown menu showing 'Engineering'), 'Location' (a dropdown menu showing 'Hyderabad'), 'Salary (Annual \$)' (with '90000' entered), and 'Experience Level' (a dropdown menu showing 'Entry Level'). Below these is a 'Skills Required (comma separated)' field with the text 'React.js, JavaScript, HTML, CSS, Node.js, Git'. A 'Job Description' field contains the text: 'We are looking for a passionate Backend Developer to join our development team. The candidate will be responsible for building responsive and user-friendly web applications using modern backend technologies.' At the bottom of the form are two buttons: 'Save Job Posting' and 'Cancel'. On the right side of the screen, there is a 'CareerFlow Assistant' chat window with a message: 'Hi there! I'm your virtual assistant. Ask me about jobs, login, or applying.' and a 'Type a message...' input field.

Figure 4.4: Details for Job Application

4.5 Module 3: Analytics and Monitoring Module

The analytics and monitoring module provides dashboard analytics, reports, notifications, login tracking, and security monitoring features. Interactive charts and graphical reports are generated using Chart.js to visualize placement statistics, application analytics, and recruitment activities.

Welcome Back

Email

yamini3366@gmail.com

Password

.....

Sign In

Don't have an account? [Register](#)

Figure 4.5: Login details

Advantages of this Project

a) Secure Authentication:

JWT authentication and bcrypt password hashing ensure secure login systems and protected access control for both administrators and students.

b) Centralized Recruitment Management:

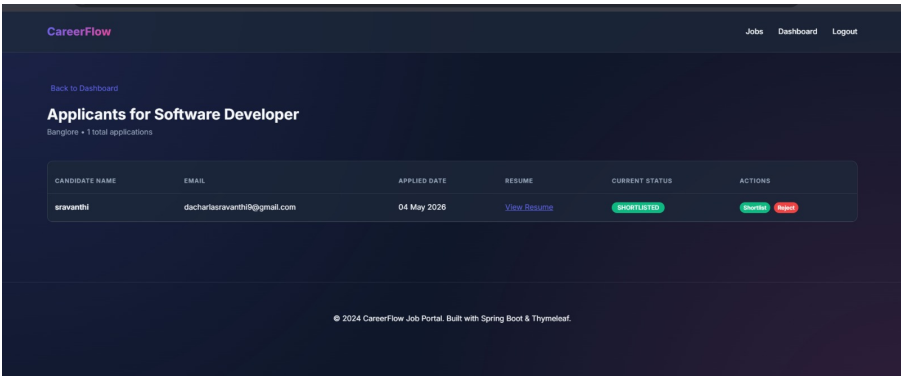
The platform allows centralized management of students, companies, jobs, applications, notifications, and reports through a single integrated system.

c) Analytics and Dashboard Systems:

Interactive dashboards and graphical analytics provide better monitoring of placement activities, recruitment statistics, and application tracking.

Disadvantages

The platform requires continuous internet connectivity for accessing online features and recruitment services. Initial setup and deployment may require technical knowledge related to full stack development, database configuration, and server management. Large-scale deployment may also require cloud infrastructure and advanced security optimization for handling high user traffic efficiently.



The screenshot displays the 'CareerFlow' application interface. At the top, there's a navigation bar with 'Jobs', 'Dashboard', and 'Logout' links. Below this, a 'Back to Dashboard' link is visible. The main heading is 'Applicants for Software Developer', followed by a sub-heading 'Bangalore • 1 total applications'. A table lists the applicants with columns for Candidate Name, Email, Applied Date, Resume, Current Status, and Actions. One applicant, 'sravanthi', is listed with an email 'dacharlasravanthi@gmail.com', applied on '04 May 2026', and a 'View Resume' link. The 'Current Status' is 'REJECTED' (indicated by a red pill button), and the 'Actions' column contains 'Reserve' and 'Reject' buttons. The footer shows '© 2024 CareerFlow Job Portal. Built with Spring Boot & Thymeleaf.'

CANDIDATE NAME	EMAIL	APPLIED DATE	RESUME	CURRENT STATUS	ACTIONS
sravanthi	dacharlasravanthi@gmail.com	04 May 2026	View Resume	REJECTED	<button>Reserve</button> <button>Reject</button>

Figure 4.6: Students Applied

Chapter 5

RESULTS AND DISCUSSIONS

The Job Portal Management System: Enterprise Recruitment & Placement Platform was successfully designed and implemented using React.js, Node.js, Express.js, and Firebase. The platform provides secure authentication, recruitment analytics, company management, job application tracking, dashboard systems, notifications, and login monitoring functionalities. The frontend interface was developed with responsive UI components and modern dashboard layouts to provide an interactive user experience for both students and administrators.

The Student Dashboard allows users to browse job opportunities, apply for jobs, track applications, view notifications, and manage profiles efficiently. The Admin Dashboard provides centralized management of students, companies, jobs, analytics, reports, and recruitment activities.

The Firebase database successfully stores and manages students, companies, jobs, applications, notifications, and login tracking information. JWT authentication and bcrypt password hashing ensure secure login functionality and protected access control across the platform.

The project demonstrates scalable architecture, responsive design, secure authentication, and enterprise-level recruitment management suitable for real-world placement systems.

Table 5.1: Comparison of Recruitment Platform Features

Applications	Advantages
Traditional Placement System	Manual Operations
Basic Recruitment Portals	Limited Analytics
LinkedIn	Professional Networking
Internshala	Internship Management
Job Portal Management System	Secure Analytics Dashboard

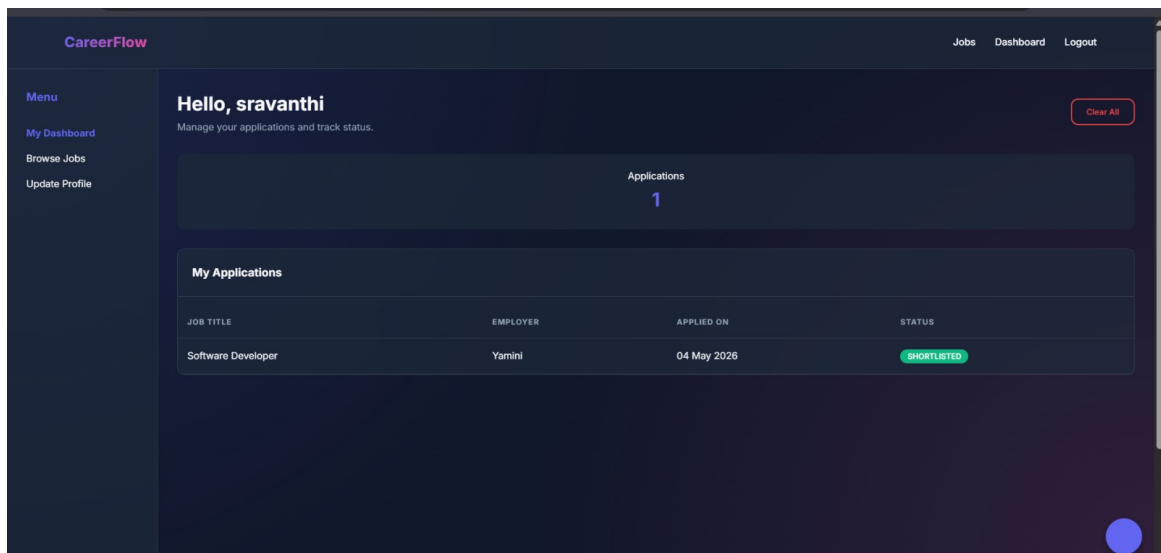


Figure 5.1: Students Shortlisted

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Management System: Enterprise Recruitment & Placement Platform was successfully designed and developed as a modern full stack recruitment management system. The platform provides a centralized environment for managing student placements, company recruitment activities, job applications, analytics, and dashboard systems. The project integrates secure authentication using JWT and bcrypt, responsive user interfaces, recruitment analytics, notifications, login tracking, and role-based access control to improve the efficiency and security of placement management.

The Student Dashboard enables students to explore job opportunities, apply for jobs, track applications, and manage their profiles through an interactive and user-friendly interface. The Admin Dashboard provides centralized control for managing students, companies, job postings, reports, analytics, and security monitoring features. The use of React.js, Node.js, Express.js, and Firebase ensures scalability, maintainability, and efficient database operations within the platform.

The project demonstrates modern web development concepts, secure authenti-

cation mechanisms, responsive dashboard systems, and enterprise-level recruitment management suitable for real-world educational institutions and placement environments.

6.2 Future Enhancements

The Job Portal Management System can be further enhanced with several advanced features to improve scalability and user experience. Future improvements may include AI-based job recommendation systems, mobile application support, cloud-native deployment, real-time notifications, advanced analytics dashboards, and video interview integration. Additional features such as resume parsing, chatbot assistance, company ranking systems, and online assessment modules can also be integrated into the platform to provide a more advanced recruitment ecosystem.

The platform can also be expanded using cloud technologies and scalable microservice architectures to support large-scale enterprise deployment and high user traffic efficiently.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Complex Engineering Problem (CEP)

Level	Attribute	Justification
WP1	Depth of Knowledge	Required of React.js, Node.js, MySQL, authentication systems, and dashboard analytics.
WP2	Conflicting Requirements	Balances secure authentication, responsive UI, analytics systems, and recruitment workflows.
WP3	Depth of Analysis	Includes API development, database design, authentication logic, dashboard implementation.
WP4	Familiarity	Recruitment analytics and login tracking introduce modern engineering challenges.
WP5	Applicable Codes	Requires custom implementation for notifications, analytics, and application tracking.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Complex Engineering Activities (CEA)

EA No.	Attribute	Justification
EA1	Range of Resources	Uses React.js, Tailwind CSS, Node.js, Express.js, MySQL, and JWT authentication.
EA2	Level of Interactions	Handles interactions between dashboards, authentication, analytics, and applications.
EA3	Innovation	Includes responsive dashboards, analytics systems, and secure authentication.
EA4	Impact on Society	Improves placement accessibility and digital recruitment management.
EA5	Familiarity	Combines analytics, notifications, and application tracking into one platform.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Facilitates employment opportunities by connecting students with companies and improving placement processes through a centralized recruitment platform.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in campus recruitment using modern full stack technologies, analytics dashboards, and secure authentication systems.
SDG 10	Reduced Inequalities	Provides a standardized and accessible recruitment platform for all students regardless of location or background.
SDG 16	Peace, Justice and Strong Institutions	Ensures data security, role-based access control, login monitoring, and secure authentication to maintain fairness and transparency in recruitment activities.

Table 7.3: SDG Mapping for the Project

Chapter 8

SOURCE CODE

8.1 Source Code

```
import React, { useEffect, useState } from "react";
import {
  BrowserRouter as Router,
  Routes,
  Route,
  Navigate
} from "react-router-dom";

import Login from "../components/Login";
import Register from "../components/Register";
import Dashboard from "../components/Dashboard";
import JobList from "../components/JobList";
import JobDetails from "../components/JobDetails";
import PostJob from "../components/PostJob";
import AppliedJobs from "../components/AppliedJobs";
import RecruiterPanel from "../components/RecruiterPanel";
import Navbar from "../components/Navbar";

function App() {

  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);
  const [jobs, setJobs] = useState([]);
```

```

// Restore User Session
useEffect(() => {
  const savedToken = localStorage.getItem("jp_token");
  const savedUser = localStorage.getItem("jp_user");

  if (savedToken && savedUser) {
    setToken(savedToken);
    setUser(JSON.parse(savedUser));
  }
}, []);

// Fetch Jobs
useEffect(() => {
  if (!user) return;

  fetch("http://localhost:5000/api/jobs")
    .then((res) => res.json())
    .then((data) => {
      if (data.success) {
        setJobs(data.jobs);
      }
    })
    .catch(() => console.log("Server Error"));
}, [user]);

// Login Handler
const handleLogin = (userData, jwtToken) => {
  setUser(userData);
  setToken(jwtToken);

  localStorage.setItem("jp_token", jwtToken);
  localStorage.setItem("jp_user", JSON.stringify(userData));
};

// Logout Handler
const handleLogout = () => {
  localStorage.clear();
  setUser(null);
  setToken(null);
};

```

```

};

// Authentication Routes
if (!user) {
  return (
    <Router>
      <Routes>

        <Route
          path="/login"
          element={<Login onLogin={handleLogin} />}
        />

        <Route
          path="/register"
          element={<Register onLogin={handleLogin} />}
        />

        <Route
          path="*"
          element={<Navigate to="/login" />}
        />

      </Routes>
    </Router>
  );
}

```

```

// Main Application
return (
  <Router>

    <Navbar
      user={user}
      onLogout={handleLogout}
    />

    <Routes>

```

```

<Route
  path="/"
  element={
    <Dashboard
      user={user}
    />
  }
/>

```

```

<Route
  path="/jobs"
  element={
    <JobList
      jobs={jobs}
      user={user}
    />
  }
/>

```

```

<Route
  path="/job/:id"
  element={
    <JobDetails
      jobs={jobs}
      user={user}
      token={token}
    />
  }
/>

```

```

<Route
  path="/post-job"
  element={
    <PostJob
      token={token}
      user={user}
    />
  }
/>

```

```

    <Route
      path="/applied-jobs"
      element={
        <AppliedJobs
          token={token}
        />
      }
    />

    <Route
      path="/recruiter"
      element={
        <RecruiterPanel
          token={token}
        />
      }
    />

  </Routes>

</Router>
);
}

export default App;

```

References

- [1] LinkedIn, “Professional Networking and Recruitment Platform,” 2026. [Online]. Available: <https://www.linkedin.com>
- [2] Naukri.com, “Online Job Recruitment Platform,” 2026. [Online]. Available: <https://www.naukri.com>
- [3] Indeed, “Global Job Search and Hiring Platform,” 2026. [Online]. Available: <https://www.indeed.com>
- [4] Internshala, “Internship and Training Portal,” 2026. [Online]. Available: <https://internshala.com>
- [5] Glassdoor, “Career Opportunities and Company Reviews Platform,” 2026. [Online]. Available: <https://www.glassdoor.com>
- [6] Monster, “Online Employment Solution Platform,” 2026. [Online]. Available: <https://www.monster.com>
- [7] React.js, “Frontend JavaScript Library Documentation,” 2026. [Online]. Available: <https://react.dev>
- [8] Node.js, “JavaScript Runtime Environment Documentation,” 2026. [Online]. Available: <https://nodejs.org>
- [9] Express.js, “Backend Web Application Framework,” 2026. [Online]. Available: <https://expressjs.com>

[10] MongoDB, “NoSQL Database Documentation,” 2026. [Online]. Available: <https://www.mongodb.com>